SHIT Sciences Humanities Information Technology

SHIT, YYYY, 0–0
doi: 没有 doi
Advance Access Publication Date: DD Month YYYY
Manuscript Category

Machine Learning

俺寻思之力的可靠性分析：工程直觉语言在 CI 成败预测中的增量信息量评估与偏差校正

可燃乌龙茶*,Jizhi FishNest,魔都寻思者,狗懒子

弗雷尔卓德大学

Associate Editor:可燃乌龙茶(dy54492591731)

摘要

工程实践中，“俺寻思之力”(should work/probably/I think)等直觉性措辞常被用来表达对改动正确性的信念，但其是否具有可迁移的预测价值缺乏量化证据。本文基于 GitHub 公共仓库 PR(Pull Request)构建数据集，并以 CI 成败作为客观真值；CI 的工程语义与实践背景遵循持续集成的经典定义与实践框架[1-2]。在数据获取层面，CI 真值优先由 GitHub check runs 体系获得，缺失时回退到 commit status 体系[3-4]。在“首次 CI 完成时间”之前提取直觉语言特征，并结合改动规模与流程强度等客观证据特征，采用可解释的逻辑回归框架，在时间切分与跨仓库(repo-split)设置下评估语言信号的增量信息量。结果表明，直觉语言对 CI 成败的增量贡献接近于零，跨仓库重复实验中ΔAUC 均值为−0.002(区间跨 0)，提示其更像语境与表达风格而非可迁移工程证据。将任务转化为失败预警更具工程意义：repo-split 下 AUC_fail 约为 0.65，且风险 top-10% PR 可覆盖约 18%的失败；此外，Sigmoid 校准能显著改善概率输出的可靠性。总体而言，在纯机器判据下的直觉语言不宜单独作为决策证据，更合理的用法是与客观证据结合并经校准后用于风险筛查与资源调度。

关键词：工程直觉；自然语言信号；持续集成；CI 失败预警；可迁移性；概率校准；GitHub

1 引言

在软件与系统工程中，技术决策常伴随不确定性。开发者在提交 PR、请求 review 或解释改动时，往往会使用“should work”“I think”“probably”等直觉性措辞传递对结果的信心或不确定性。持续集成(CI)作为工程过程控制手段，常被用于在自动化构建与测试流水线上给出相对客观的成败反馈，其概念与实践框架可追溯到经典 CI 论述与工程化总结[1-2]。然而，CI 成败受多种因素影响(配置、依赖、测试、环境等)，且在开源与企业环境中都被观察到系统性失败模式与类别分布[5][7-8]；围绕 CI 数据的整合与可复现实证研究也形成了公开数据基础[6](如 Travis 生态的汇聚数据)。因此，一个看似简单但长期被忽视的问题是：直觉语言能否作为可量化证据，独立预测 CI 成败？它是否具有跨项目可迁移性？

如果俺寻思之力确实携带信息，它可能用于：提前筛查高风险 PR，优化 review/CI 资源分配；在缺乏完整测试证据时提供辅助决策信号；通过校准将语言信号转化为可用概率，支持阈值策略[9][11-12]。

反之，如果直觉语言不具备可迁移信息量，或在跨仓库设置下引入负迁移，则应谨慎使用，避免将“措辞风格”误当作“工程证据”[10][13]。

本文的贡献如下：构建了一个以 CI 成败为真值的 PR 数据集，并通过对照采样缓解“仅关键词样本”的选择偏差；提出严格的时间截断策略，防止利用 CI 结果之后的讨论文本造成信息泄漏；在时间切分与 repo-split(跨仓库泛化)两种评估协议下，量化直觉语言的增量信息量；将任务转化为失败预警，给出 top-K 风险筛查的可解释工程指标；展示概率校准对“可用概率输出”的必要性与收益。

2 数据与任务定义

2.1 数据来源与采样策略

研究对象为 GitHub 公共仓库中已关闭 PR。为了获得包含直觉语言的样本并控制工作量[14-15]，本文首先通过检索 PR 讨论文本中的触发短语(如“should work”“I think”“probably”等)定位候选 PR，构成 case 集。然而仅使用触发词检索会导致缺失对照组，从而无法回答“语言是否提供增量信息”。因此本文进一步构建 control 集：在与每个 case 相同仓库与时间窗内，随机采样已关闭 PR，不要求包含任何触发词[16-18]。

最终数据集规模：case：3000 条(由触发词定位)control：1275 条(同仓库同时间窗随机采样)总计：4275 条 PR；覆盖约 2100 个仓库量级(以实际统计为准)

2.2 CI 真值与防泄漏截断

本文以 CI 结局作为客观真值($Y\in\{0,1\}$)：success 为 1，fail 为 0。CI 结局优先从 GitHub 的 check runs 体系获得；若缺失，则回退到 commit status 体系[3-4]。

为避免信息泄漏，定义每个 PR 的截断时间为“首次 CI 完成时间”(最早的 completed check run 时间，或最早终态 status 时间)。语言特征仅从截断时间之前的 PR 标题、正文与讨论评论中提取；截断之后的“修 CI/改完了/通过了”等文本不纳入特征。

2.3 任务 1：成功预测(success as positive)

目标是估计

$$P(Y_{success} = 1|I, E) \tag{1}$$

并比较 I-only、E-only、I+E 的性能差异，尤其关注增量信息量：

$$\Delta AUC=AUC(I+E)-AUC(E) \tag{2}$$

2.4 任务 2：失败预警(fail as positive)

工程场景更关心失败，因此定义 $Y_{fail}=1-Y_{success}$，并用风险分数

$$S_{fail}=1-P(Y_{success}=1) \tag{3}$$

评估 AUC_fail、AP_fail 以及 Recall@topK：挑选风险最高的 top-K%PR，覆盖多少真实失败[22-23]。

3 特征设计与模型

3.1 直觉语言特征(I)

为保证跨样本一致性并避免文本长度估计差异引入偏差，本文主要采用 lex-only 方案，笔者曾尝试过密度特征与 has_*事件特征，但由于 case/control 的 token 统计口径不一致，会引入偏差[10-13]；最终主结果以 lex-only 为准：

lex_confident：肯定类词计数(如 definitely/for sure/will work 等)

lex_probable：可能类/直觉类词计数(should work/probably/I think/seems 等)

lex_uncertain：不确定/风险类词计数(not sure/might fail/may break 等)

3.2 客观证据特征(E)

E 反映改动规模与流程强度，使用：additions、deletions、changed_files、commits、n_comments_pre_ci：取 log1p 以缓解长尾；n_check_runs：CI 检查项数量，其中流程复杂度与失败概率相关[7-9]。

3.3 模型与训练

使用标准化+逻辑回归作为可解释基线：

StandardScaler 对输入特征标准化；LogisticRegression，class_weight=balanced，以缓解类别不均衡对决策阈值的偏置[22-23]；评估指标：AUC、AP、Brier、log-loss、accuracy@0.5。

3.4 概率校准

The quick 由于 class_weight=balanced 与特征标准化会导致概率尺度偏移，本文在验证集上使用 Sigmoid(Platt scaling)进行校准，并在测试集评价 Brier 与 log-loss 改善。

4 实验设置

4.1 时间切分(time split)

按 created_at 时间排序后切分 70/15/15(train/val/test)，模拟“用过去预测未来”。

4.2 跨仓库泛化(repo split)

使用 GroupShuffleSplit 以 repo 为分组单位切分 train/test，确保测试仓库不在训练集出现。并进一步重复 30 次不同随机种子，报告均值、标准差与分位数区间，避免单次划分偶然性。

4.3 Bootstrap 不确定性

在固定模型输出下，对测试集进行 bootstrap 重采样，估计指标(如ΔAUC)的经验分布与 95%区间[25]。

5 实验结果

5.1 语言增量信息量：时间切分下接近 0

在时间切分设置中，E-only 已提供主要判别能力；I-only 近似随机；I+E 相对 E-only 的增量非常小，ΔAUC 约为千分级，95%区间跨 0，表明在同一数据源时间演进下，直觉语言并不提供稳定的额外信息(见图 1、图 2)。

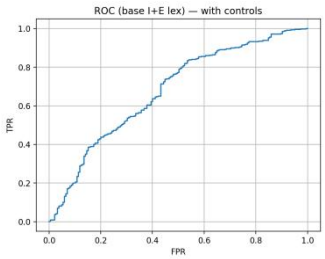

图 1 时间切分(time-split)下 I+E(lex-only)模型的 ROC 曲线

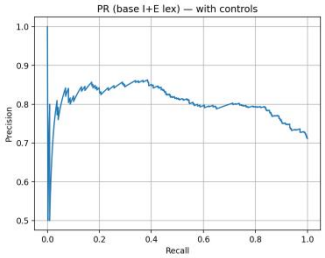

图 2 时间切分(time-split)下 I+E(lex-only)模型的 PR 曲线

5.2 repo-split 重复实验：语言无迁移增益，整体轻微负增量

跨仓库泛化评估表明，加入语言特征后模型性能的增量在统计意义上无法与零区分，且平均变化呈轻微负向。这意味着该类直觉性措辞更可能反映不同仓库的协作语境、沟通习惯与表述策略，而非稳定可迁移的工程证据：同一短语在不同项目中可能对应不同程度的谨慎、承诺或礼貌性表达，从而使语言信号在跨项目场景下难以提供一致、可靠的预测增益。

表 1 30 次 repo-split 重复实验汇总结果

ΔAUC mean=−0.002001，std=0.004271， p2.5–p97.5=[−0.011888,0.003412] ΔAP mean=−0.001597，p2.5–p97.5=[−0.011337,0.004606]
ΔBrier mean=0.000115(区间跨 0)
ΔLogLoss mean =0.000271(区间跨 0)

5.3 失败预警的工程价值

在 repo-split 的重复实验设置下，将任务定义为“失败预警”(以 fail 为正类)后，模型展现出稳定的风险筛查能力：平均 AUC_fail 为 0.645(2.5%–97.5%分位为 0.577–0.712)，平均 AP_fail 为 0.437(0.315–0.576)，说明模型在跨仓库泛化条件下对失败样本具有中等强度的区分与排序能力(见图 3、图 4)。进一步从资源调度视角考察 top-K 风险覆盖率，若仅对风险最高的 5% PR 进行重点审查，平均可覆盖约 9.96%的失败；将审查比例提升至 10%与 20%时，覆盖率分别提高到约 18.25%与 33.17%(对应区间分别为 0.1327–0.2513 与 0.2737–0.4027)。这表明该模型更适合作为工程流程中的“风险筛查器”：在审查资源受限的情况下，通过优先处理高风险 PR，可以以较小的审查比例换取对失败事件的显著覆盖，从而降低 CI 返工与排队成本。

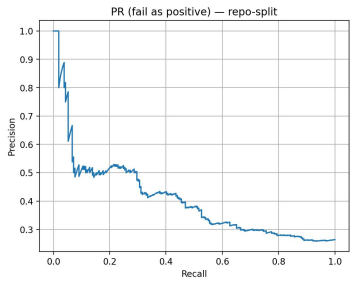

图 3 repo-split 下失败为正类的 PR 曲线

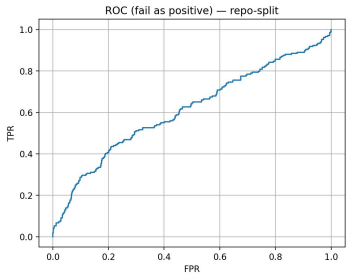

图 4 repo-split 下失败为正类的 ROC 曲线

5.4 概率校准对可用性的显著改善

在 time-split(with controls，lex-only I+E)设置下，Sigmoid 校准显著降低 Brier 与 log-loss。因此，尽管排序能力提升有限，校准后的概率输出更适合阈值决策。

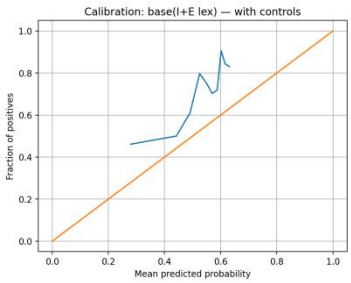

图 5 时间切分下未校准概率的可靠性曲线

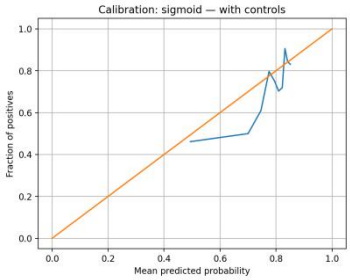

图 6 时间切分下 Sigmoid 校准后的可靠性曲线

6 讨论

6.1 为什么直觉语言不提供可迁移增益

直觉语言的两个关键属性导致其在 repo-split 下难以泛化[10][13]：

语境依赖性：不同项目对“should work”的使用语义不同；可能是谨慎承诺，也可能是口头禅或礼貌性措辞；

表达风格混杂：是否使用直觉词受个体写作习惯影响，不等价于工程能力或实际风险。

因此，在不引入更强语义模型与更严格因果控制的情况下，lex-only 信号更像“风格变量”，对结果的贡献被 E 吸收或表现为噪声。

6.2 为什么失败预警更具有价值

工程流程中的关键目标不是提高“成功 PR 的置信”，而是减少失败带来的迭代开销。失败预警的 top-K 策略能与真实流程对齐：用有限审查资源覆盖尽可能多的失败，从而降低 CI 排队与返工成本。本文结果表明，即便模型仅达到中等区分度，也能提供可用的筛查收益[7-8]。

6.3 校准的重要性

在许多实际系统中，决策依赖概率阈值而不仅是排序。class_weight 与数据不均衡会导致概率偏移，校准是将模型输出转化为“可信成功率/失败率估计”的必要步骤。Sigmoid 校准在本数据上表现更稳健。

7 局限性

语言特征为词汇计数，无法识别反讽、否定、上下文与复杂语义；control 集规模受 API 限制，小于 case；尽管足以构造对照，但仍可能存在仓库规模偏差；CI 真值仅为 success/fail，不区分失败原因(测试失败、lint、超时、权限等)；文本语言以英文短语为主，非英文仓库可能被低估或排除；这是观察性研究，无法做严格因果推断。

8 结论

本文通过构建包含对照组的 GitHub PR 数据集，并在防泄漏特征提取、时间切分与跨仓库泛化评估下，量化了直觉语言对 CI 成败预测的增量信息量。结果表明：直觉语言(lex-only)对 CI 成败缺乏稳定可迁移增益，增量指标在多次 repo-split 下均值接近 0 且略偏负；但以失败预警为目标时，模型可作为风险筛查器提供可量化的资源调度收益。概率校准能显著提高输出概率的可用性。整体而言，“俺寻思之力”(should work/probably/I think)等直觉性措辞更适合被视为沟通风格而非工程证据；工程流程更应依赖可量化的客观证据，并用校准后的风险分数支持决策。

9 数据集及运行环境

原始 case 数据、合并数据集：联系作者畅饮可燃乌龙茶后获取

运行环境：Python+scikit-learn；GitHub REST API(checks/status)

参考文献

[1] Fowler M. Continuous Integration[EB/OL]. 2024-01-18(更新)[2026-02-25 访问].

[2] Duvall P M, Matyas S, Glover A. Continuous Integration: Improving Software Quality and Reducing Risk[M]. Addison-Wesley Professional, 2007. ISBN: 978-0321336385.

[3] GitHub Docs. REST API endpoints for check runs[EB/OL]. [2026-02-25 访问].

[4] GitHub Docs. REST API endpoints for commit statuses[EB/OL]. [2026-02-25 访问].

[5] Beller M, Gousios G, Zaidman A. Oops, My Tests Broke the Build: An Explorative Analysis of Travis CI with GitHub[C]//Proceedings of the 14th International Conference on Mining Software Repositories (MSR 2017). 2017: 356-367. DOI:10.1109/MSR.2017.62.

[6] Beller M, Gousios G, Zaidman A. TravisTorrent: Synthesizing Travis CI and GitHub for Full-Stack Research on Continuous Integration[C]//Proceedings of MSR 2017. 2017: 447-450. DOI:10.1109/MSR.2017.24.

[7] Rausch T, Hummer W, Leitner P, Schulte S. An Empirical Analysis of Build Failures in the Continuous Integration Workflows of Java-Based Open-Source Software[C]//MSR 2017. 2017: 345-355. DOI:10.1109/MSR.2017.54.

[8] Vassallo C, Schermann G, Zampetti F, et al. A Tale of CI Build Failures: An Open Source and a Financial Organization Perspective[C]//2017 IEEE International Conference on Software Maintenance and Evolution (ICSME 2017). 2017: 183-193. DOI:10.1109/ICSME.2017.67.

[9] Saidani I, Ouni A, Chouchen M, Mkaouer M W. Predicting continuous integration build failures using evolutionary search[J]. Information and Software Technology, 2020, 128: 106392. DOI:10.1016/j.infsof.2020.106392.

[10] Al-Sabbagh K W, Staron M, Hebig R. Predicting Build Outcomes in Continuous Integration using Textual Analysis of Source Code Commits[C]//PROMISE ’22. 2022: 42-51. DOI:10.1145/3558489.3559070.

[11] Santolucito M, Zhang J, Zhai E, Cito J, Piskac R. Learning CI Configuration Correctness for Early Build Feedback[C]//2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER 2022). 2022: 1006-1017. DOI:10.1109/SANER53432.2022.00118.

[12] Hong Y, Tantithamthavorn C, Pasuksmit J, et al. Practitioners’ Challenges and Perceptions of CI Build Failure Predictions at Atlassian[C]//Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion ’24). 2024. DOI:10.1145/3663529.3663856.

[13] Santos E A, Hindle A. Judging a Commit by Its Cover: Correlating Commit Message Entropy with Build Status on Travis-CI[C]//Proceedings of the 13th International Conference on Mining Software Repositories (MSR 2016). DOI:10.1145/2901739.2903493.

[14] Gousios G, Pinzger M, van Deursen A. An Exploratory Study of the Pull-Based Software Development Model[C]//Proceedings of the 36th International Conference on Software Engineering (ICSE 2014). DOI:10.1145/2568225.2568260.

[15] Tsay J, Dabbish L, Herbsleb J. Influence of Social and Technical Factors for Evaluating Contribution in GitHub[C]//ICSE 2014. 2014: 356-366.

[16] Yu Y, Wang H, Filkov V, Devanbu P T, Vasilescu B. Wait for It: Determinants of Pull Request Evaluation Latency on GitHub[C]//MSR 2015. 2015: 367-371. DOI:10.1109/MSR.2015.42.

[17] Lenarduzzi V, Nikkola V, Saarimäki N, Taibi D. Does code quality affect pull request acceptance? An empirical study[J]. Journal of Systems and Software, 2021, 171: 110806. DOI:10.1016/j.jss.2020.110806.

[18] Ortu M, Destefanis G, Graziotin D, Marchesi M, Tonelli R. How Do You Propose Your Code Changes? Empirical Analysis of Affect Metrics of Pull Requests on GitHub[J]. IEEE Access, 2020. DOI:10.1109/ACCESS.2020.3002663.

[19] Platt J. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods[C]//Advances in Large Margin Classifiers. 1999.

[20] Niculescu-Mizil A, Caruana R. Predicting Good Probabilities with Supervised Learning[C]//Proceedings of the 22nd International Conference on Machine Learning (ICML ’05). 2005.

[21] Zadrozny B, Elkan C. Transforming Classifier Scores into Accurate Multiclass Probability Estimates[C]//Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’02). 2002. DOI:10.1145/775047.775151.

[22] Hanley J A, McNeil B J. The meaning and use of the area under a receiver operating characteristic (ROC) curve[J]. Radiology, 1982.

[23] Davis J, Goadrich M. The Relationship Between Precision-Recall and ROC Curves[C]//Proceedings of the 23rd International Conference on Machine Learning (ICML 2006). 2006.

[24] Brier G W. Verification of Forecasts Expressed in Terms of Probability[J]. Monthly Weather Review, 1950, 78(1): 1-3. DOI:10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[25] Efron B, Tibshirani R J. An Introduction to the Bootstrap[M]. Chapman & Hall/CRC, 1993. France.